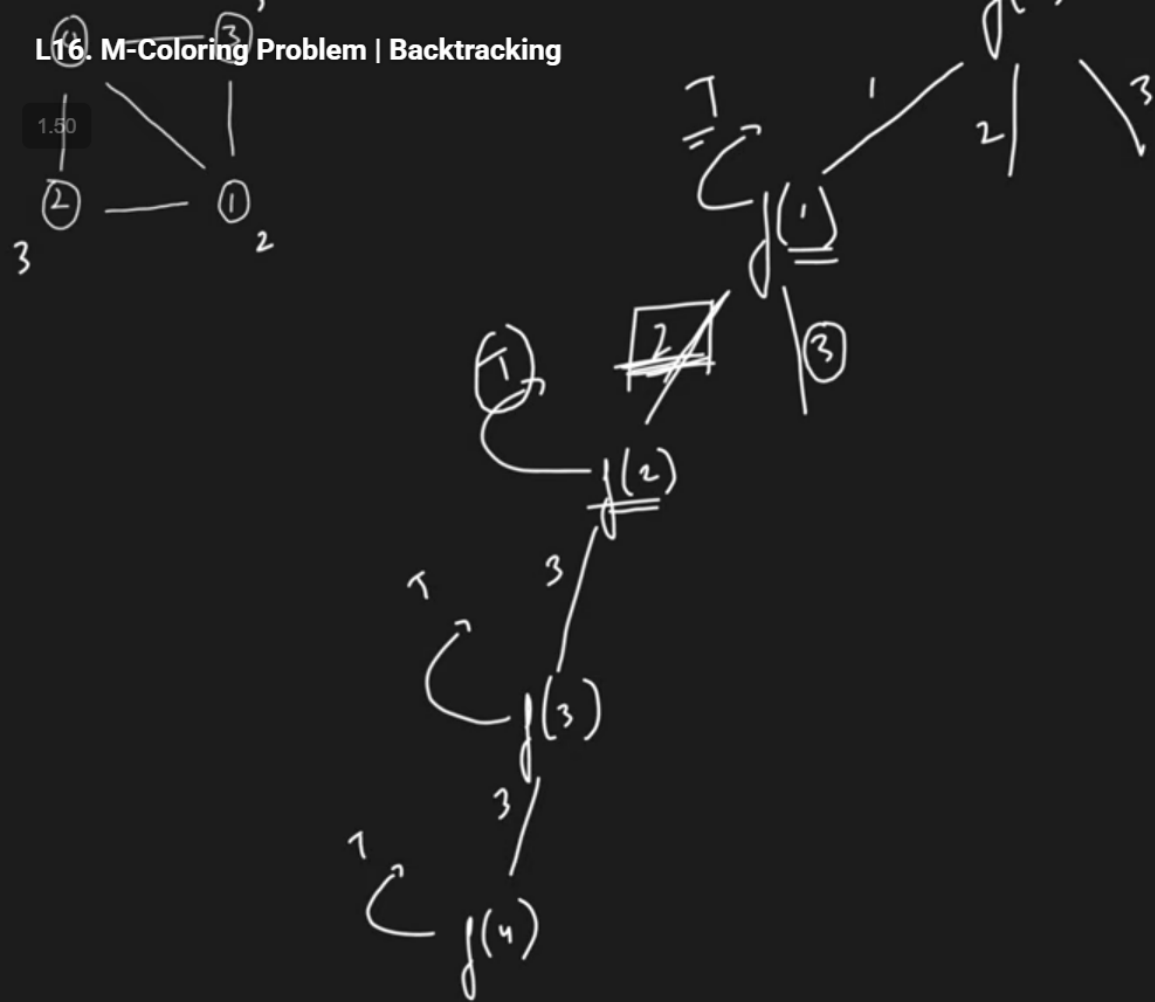


L16. M-Coloring Problem | Backtracking

1.50



$f(\underline{\text{node}})$

$\{ \text{if}(\text{node} == N) \text{ return } T; \}$

$\text{for}(\underline{\text{col}} = (1 \rightarrow m))$

$\{ \text{if}(\text{possible} \rightarrow \checkmark) \}$

$\{ \text{color}[\text{node}] = \text{col}; \}$

$\text{if}(\text{f}(\text{node} + 1) == T)$

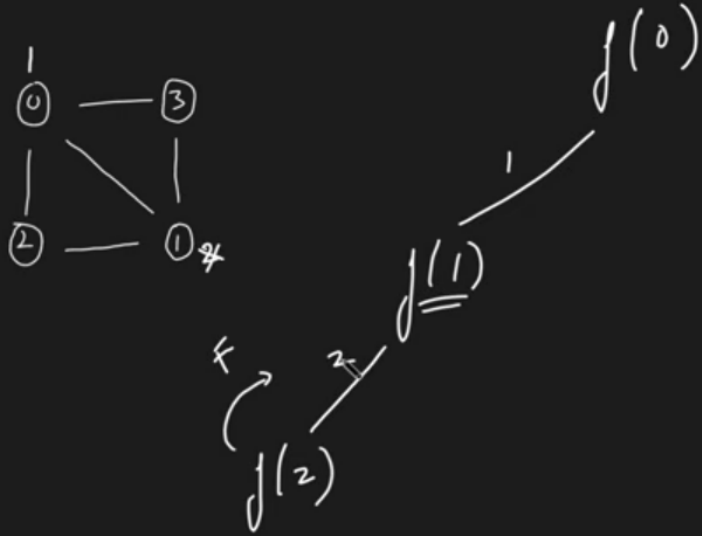
$\text{return } T;$

$\{ \text{color}[\text{node}] = 0; \}$

$\{ \text{return } F; \}$

L16. M-Coloring Problem | Backtracking

1.50



$$\boxed{M=3^2}$$

$$\underline{\underline{N=4}}$$

$f(\underline{\text{node}})$
 $\{$
 if (node == N) return T;

for (col = 1 → m)
 $\{$
 if (possible → ✓)

$\{$
 color[node] = col; ✓ F

if (f(node + 1) == T)

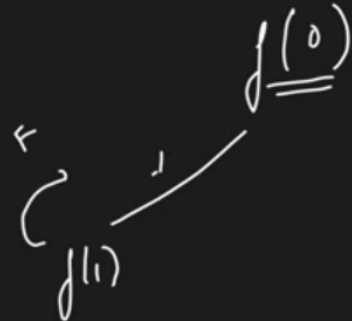
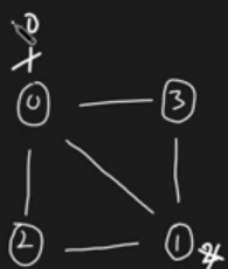
$\{$
 return T;

$\{$
 color[node] = 0;

$\{$
 return F;

L16. M-Coloring Problem | Backtracking

1.50



$$M = 3^2$$

$$N = 4$$

f(node)

{ if (node == N) return T;

for (col = 1 → m)

{ if (possible → ✓)

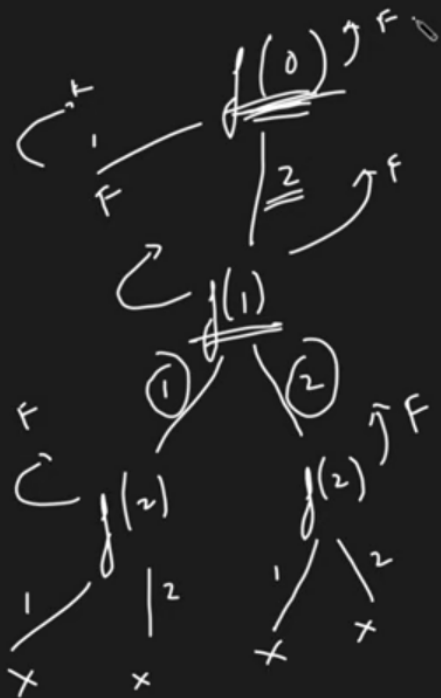
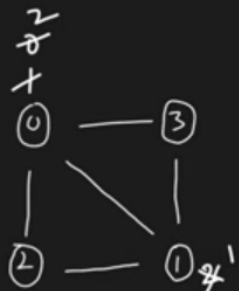
{ color[node] = col; ✓ F

if (f(node+1) == T)

return T;

{ color[node] = 0;

{ return F;



$$M = 3^2$$

$$N = 4$$

$f(\text{node})$

{
if (node == N) return T;

for (col = 1 → m)

{
if (possible → ✓)

{
color[node] = col; ✓
F

if (f(node + 1) == T)

return T;

{
color[node] = 0;

→ return F;

L16. M-Coloring Problem | Backtracking

```
10
11 bool isSafe(int node, int color[], bool graph[101][101], int m, int N, int i) {
12     for(int k = 0; k < m; k++) {
13         if(k != node && graph[k][node] == 1 && color[k] == i) {
14             return false;
15         }
16     }
17     return true;
18 }
19 bool solve(int node, int color[], int m, int N, bool graph[101][101]) {
20     if(node == N) {
21         return true;
22     }
23
24     for(int i = 1; i <= m; i++) {
25         if(isSafe(node, color, graph, N, i)) {
26             color[node] = i;
27             if(solve(node+1, color, m, N, graph)) return true;
28             color[node] = 0;
29         }
30     }
31     return false;
32 }
33
34 //Function to determine if graph can be coloured with at most M colours such
35 //that no two adjacent vertices of graph are coloured with same colour.
36 bool graphColoring(bool graph[101][101], int m, int N)
37 {
38     int color[N] = {0};
39     if(solve(0, color, m, N, graph)) return true;
40     return false;
41 }
42
43 Driver Code Ends
```

www.youtube.com - to exit full screen, press Esc